

Teamwork — Session 5: a second, independent endpoint

Context

Work in your team. The class splits into **six teams of four students**.

In class you added `GET /api/opportunities/:id` to the real backend — a route that reads `req.params.id`, looks up one record, and answers `200` with the record or `404` if nothing matches. Community posts have no equivalent endpoint yet.

Goal

Independently build the same shape of endpoint, for a different, real resource this app already has.

What to build

1. `GET /api/posts/:id`, in `server/index.js`, near the existing `GET /api/posts` route:
 - reads the id from the URL;
 - looks it up in the real in-memory `posts` array;
 - responds `200` with the full post object if found;
 - responds `404` with a clear error message if not.
2. **Prove it works, independently of the frontend** — with curl, Postman, or Thunder Client, for both a real post id and a made-up one. Nothing under `client/` needs to change for this task.

Rules

Allowed — everything from today's live coding: `app.get`, `req.params`, `res.status(...).json(...)`.

Not yet (do not use these) — a database, `req.query`, `req.body`, any new npm package, any new middleware beyond the logging one already in the file.

Do not modify `GET /api/opportunities/:id` or the request-logging middleware — this task only adds one new route.

Test it

- A real post id (check `server/data.js` for one, or use `GET /api/posts` to see all of them) returns the full post object with status `200`.
- A made-up id (e.g. `99999`) returns a clear error message with status `404`, not a crash and not an empty `200`.
- `node --check index.js` (from `server/`) reports no syntax errors.

Done when

- `GET /api/posts/:id` exists and behaves as described above.
- Both the found and not-found cases are actually tested with a real tool (curl/Postman/Thunder Client), not just read from the code.
- `node --check index.js` passes.
- Your team's attempt is committed and pushed to your own branch.

Working time

About **25 minutes**, then the official solution is demonstrated.

Before the solution is shown

Commit and push your team's attempt to your own `student-<your-github-username>` branch **before** the official solution is demonstrated. Pushing your own attempt first is part of the exercise.

Be ready to answer

- Where, in the URL, does this handler actually get the id it looks up?
- What is the difference between "the request reached a valid route but found nothing" (this task's own 404 case) and "the request didn't match any route at all"?
- Could this same shape work for a resource this app doesn't have yet? What would have to change, and what wouldn't?